It is worth noting that PHP doesn't distinguish between indexed and associative arrays internally. Indexed arrays are still associative arrays; they just have very strict rules about the keys of indexed arrays, namely, that they are sequential decimal integers. Also worth mentioning is the fact that it is not necessary to provide explicit keys for all elements. This is perfectly legal: `$arr = ['foo' => 'bar', 'qux']`. PHP will provide sequential decimal integer keys whenever it adds an element to an array.

Multidimensional Arrays

A multidimensional array is simply an array containing more arrays. As stated earlier, array elements can be compound types, and arrays are a compound data type:

```
$multiDArr = [
    [1, 2],
    [3, 4],
];

print "Value of second element of second array is {$multiDArr[1][1]}\n";

Output:
Value of second element of second array is 4
```

Here we've used an array declaration within another array declaration. The results are self-explanatory: the first and second elements of `$multiDArr` are arrays. A common concept when discussing multidimensional arrays is "inner" and "outer." In this example, the outer array is `$multiDArr`, which contains two inner arrays. Accessing inner array elements requires a slightly more complex bracket/key notation: a key followed by a sub-key: `$multiDArr[1][1]`. The first key corresponds to the outermost array, the next key (the first sub-key) to the first inner array, and so on. Be wary of code containing many array keys chained together; unless something deeply mathematical is going on, it's usually a sign that a refactor is needed (an example of traversing tree structures with recursive function calls can be found toward the end of this chapter).

It is possible to build very complex data structures using this technique, but care should be exercised: arrays of associative arrays might be better off being refactored to arrays of objects.

ArrayObject